INTEGRATED TECHNOLOGIES (AML) LAB

(Course Code: 23UPCSC1L08)

A programming laboratory record submitted to Periyar University, Salem

In partial fulfillment of the requirements for the degree of

MASTER OF COMPUTER APPLICATIONS

By

GOPIKA D

[Reg.No:U24PG507CAP004]



DEPARTMENT OF COMPUTERSCIENCE

PERIYAR UNIVERSITY

STATE UNIVERSITY - NAAC 'A++' GRADE - NIRF RANK 94

STATE PUBLIC UNIVERSITY RANK 40

SDG INSTITUTIONS RANK BAND: 11-50

SALEM - 636 011.

(OCTOBER-2026)

CERTIFICATE

This is to certify that the Programming Laboratory entitled **"INTEGRATED TECHNOLOGIES (AML) LAB (23UPCSC1L08)"** is a bonafide record work done by Mr. /Ms. _____ Register No: _____as partial fulfillment of the requirements for the degree of Master of Computer Applications, In the Department of Computer Science, Periyar University, Salem, during the Academic Year 2026-2027.

Staff In-charge Head of the Department

Submitted for the practical examination held on……...……

Internal Examiner External Examiner

CONTENTS

S.NO	DATE	TITLE OF THE PROGRAM	PAGE NO	SIGNATURE
1.		Central Tendency Measure:Mean , Median, Mode,Measure of Dispersion: Variance, Standard Deviation		
2.		Linear Regression and Multi Linear Regression with a real datasets		
3.		Logistic Regression using sklearn		
4.		Binary Classification Model		
5.		Nearest Neighbour and Naive Bayes Algorithm		
6.		Decision tree for classification using sklearn ans its parameter tuning		
7.		K-means algorithm		
8.		Image Classifier using CNN in Tensor Flow/ Keras		
9.		Autoencoder in Tensor Flow/ Keras		
10.		Simple LSTM using Tensor Flow/ Keras.		

SOURCE CODE:

```
#Central tendency
import pandas as pd
data = pd.read_csv(
    "https://raw.githubusercontent.com/mwaskom/seaborn-
    data/refs/heads/master/iris.csv")
columns = [
    'sepal_length',
    'sepal_width',
    'petal_length',
    'petal_width']
for col in columns:
    print("\n", col)
    print("Mean =", data[col].mean())
    print("Median =", data[col].median())
    print("Mode =", data[col].mode()[0])
    print("Standard Deviation =", data[col].std())
    print("Variance =", data[col].var())
```

OUTPUT:

```
 sepal_length
Mean = 5.843333333333334
Median = 5.8
Mode = 5.0
Standard Deviation = 0.8280661279778629
Variance = 0.6856935123042505

 sepal_width
Mean = 3.0573333333333337
Median = 3.0
Mode = 3.0
Standard Deviation = 0.435866284936698
Variance = 0.1899794183445188

 petal_length
Mean = 3.7580000000000005
Median = 4.35
Mode = 1.4
Standard Deviation = 1.7652982332594667
Variance = 3.1162778523489942

 petal_width
Mean = 1.1993333333333336
Median = 1.3
Mode = 0.2
Standard Deviation = 0.7622376689603465
Variance = 0.5810062639821029
```

SOURCE CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
import matplotlib.pyplot as plt
dataset=pd.read_csv("https://raw.githubusercontent.com/selva86/datasets/master/BostonHousing.csv")
dataset.info()
dataset
X=dataset.drop('medv',axis=1)
y=dataset['medv']
x=X[['age']]

#Single Linear Regression Program
x_train, x_test, y_train, y_test=train_test_split(x, y, test_size=0.2, random_state=10)
lr=LinearRegression()
lr.fit(x_train, y_train)
result=lr.predict(x_test)
mse=mean_squared_error(y_test, result)
rmse=np.sqrt(mse)
r2=r2_score(y_test, result)
print(f"Mean Squared Error: {mse:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")
print(f"R-Sqared: {r2:.2f}")
plt.scatter(y_test, result)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--',color='red')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Regression Line')
plt.show()
```

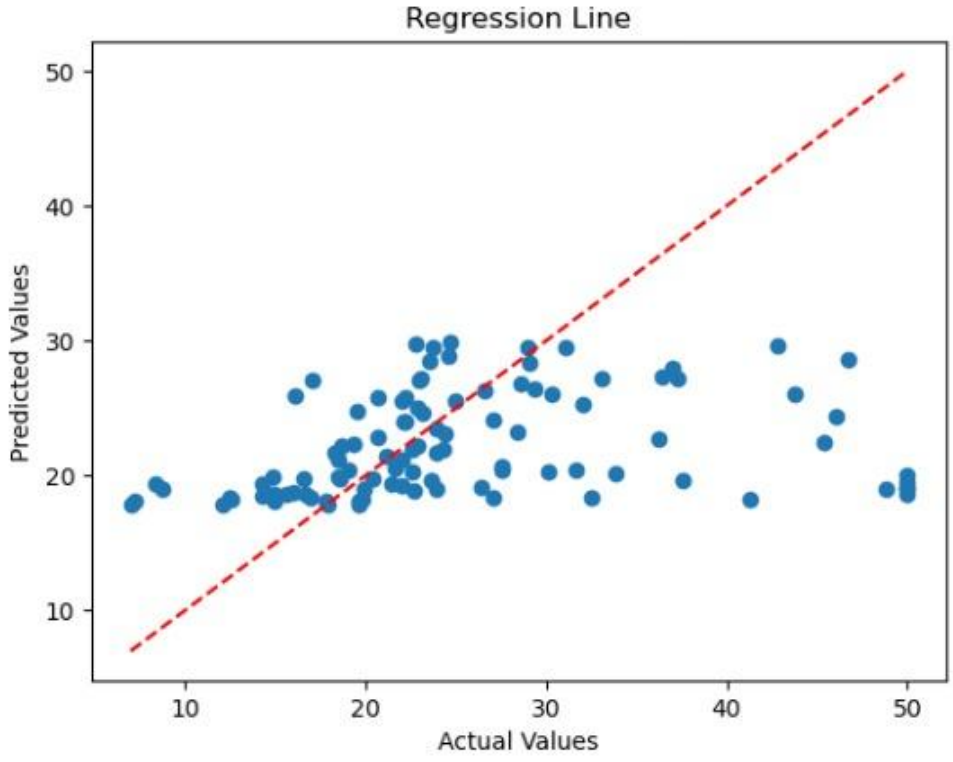
```
#Multi Linear Regression Program
x_train, x_test, y_train, y_test=train_test_split(X,y,test_size=0.2, random_state=10)
lr=LinearRegression()
lr.fit(x_train, y_train)
result1=lr.predict(x_test)
mse=mean_squared_error(y_test, result1)
rmse=np.sqrt(mse)
r2=r2_score(y_test, result1)
print(f"Mean Squared Error: {mse:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")
print(f"R-Sqared: {r2:.2f}")
plt.scatter(y_test, result1)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='-',color='red')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Regression Line')
plt.show()
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   crim     506 non-null    float64
 1   zn       506 non-null    float64
 2   indus    506 non-null    float64
 3   chas     506 non-null    int64
 4   nox      506 non-null    float64
 5   rm       506 non-null    float64
 6   age      506 non-null    float64
 7   dis      506 non-null    float64
 8   rad      506 non-null    int64
 9   tax      506 non-null    int64
 10  ptratio  506 non-null    float64
 11  b        506 non-null    float64
 12  lstat    506 non-null    float64
 13  medv     506 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
```

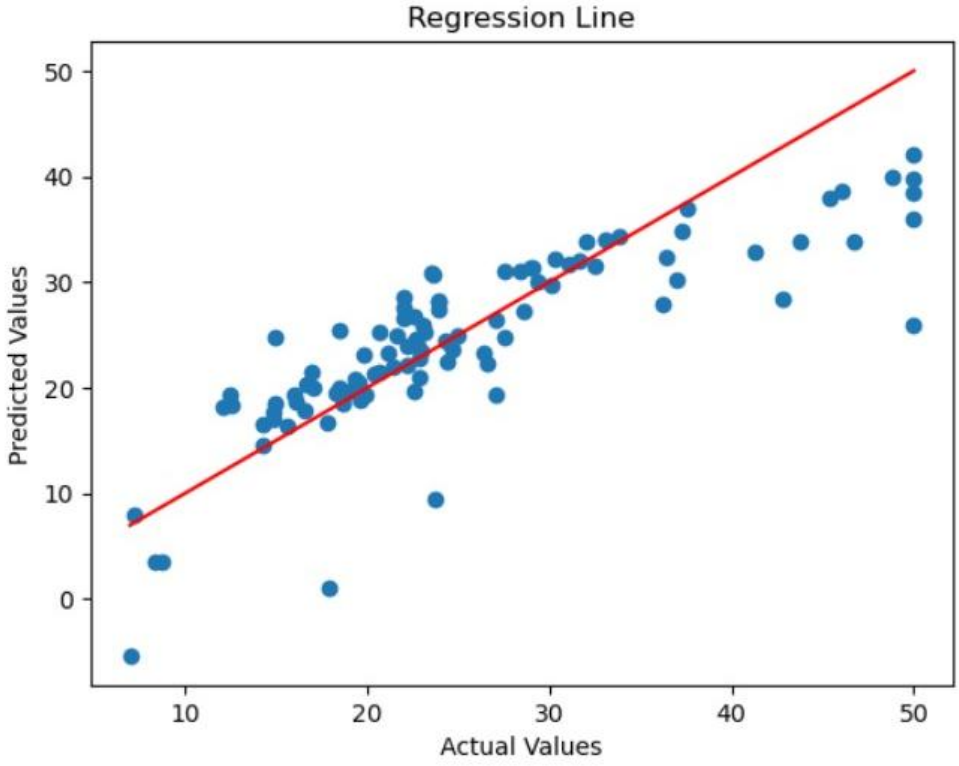
Single Linear Regression :

```
Mean Squared Error: 107.93
Root Mean Squared Error: 10.39
R-Sqared: -0.03
```



Mulitple Linear Regression :

```
Mean Squared Error: 34.41
Root Mean Squared Error: 5.87
R-Sqared: 0.67
```



SOURCE CODE:

```
# Logistic regression

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, classification_report

dataset=pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/refs/heads/master/iris.csv')

dataset.info()

x=dataset.drop('species',axis=1)

y=dataset['species']

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size= 0.2,random_state=42)

lr=LogisticRegression()

lr.fit(x_train,y_train)

result2=lr.predict(x_test)

accuracy = accuracy_score(y_test, result2)

classification_rep = classification_report(y_test, result2)

accuracy

print(classification_rep)
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

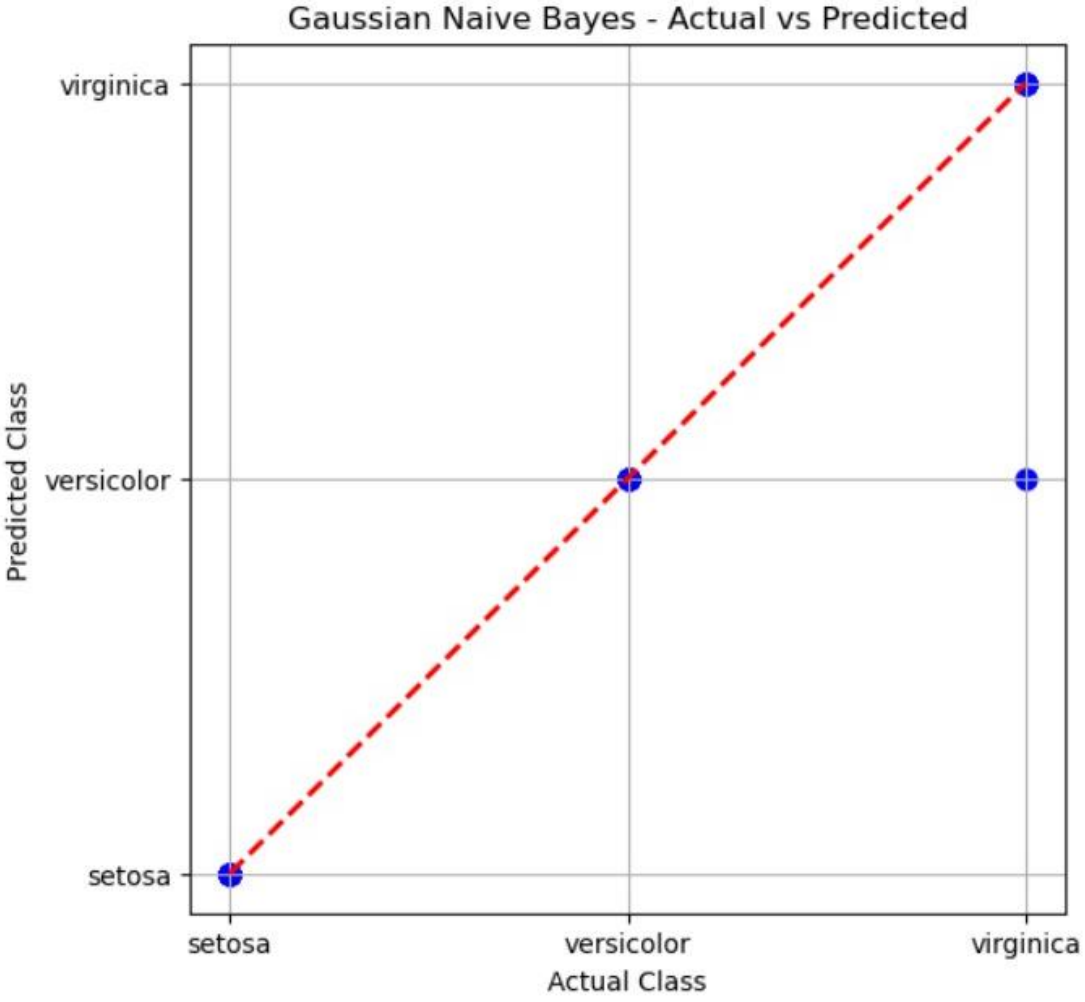
SOURCE CODE:

```
#KNN
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder
data = pd.read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv")
X = data.drop('species', axis=1)
y = data['species']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=32)
knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)
print("KNN Accuracy:", accuracy_score(y_test, knn_pred))
nb = GaussianNB()
nb.fit(X_train, y_train)
nb_pred = nb.predict(X_test)
print("Gaussian NB Accuracy:", accuracy_score(y_test, nb_pred))
le = LabelEncoder()
actual = le.fit_transform(y_test)
predicted = le.transform(nb_pred)
```

```
plt.figure(figsize=(6, 6))
plt.scatter(actual, predicted, color='blue', s=60)
plt.plot([0, 2], [0, 2], 'r--', linewidth=2)
plt.xticks([0, 1, 2], le.classes_)
plt.yticks([0, 1, 2], le.classes_)
plt.xlabel("Actual Class")
plt.ylabel("Predicted Class")
plt.title("Gaussian Naive Bayes - Actual vs Predicted")
plt.grid(True)
plt.show()
```

OUTPUT:

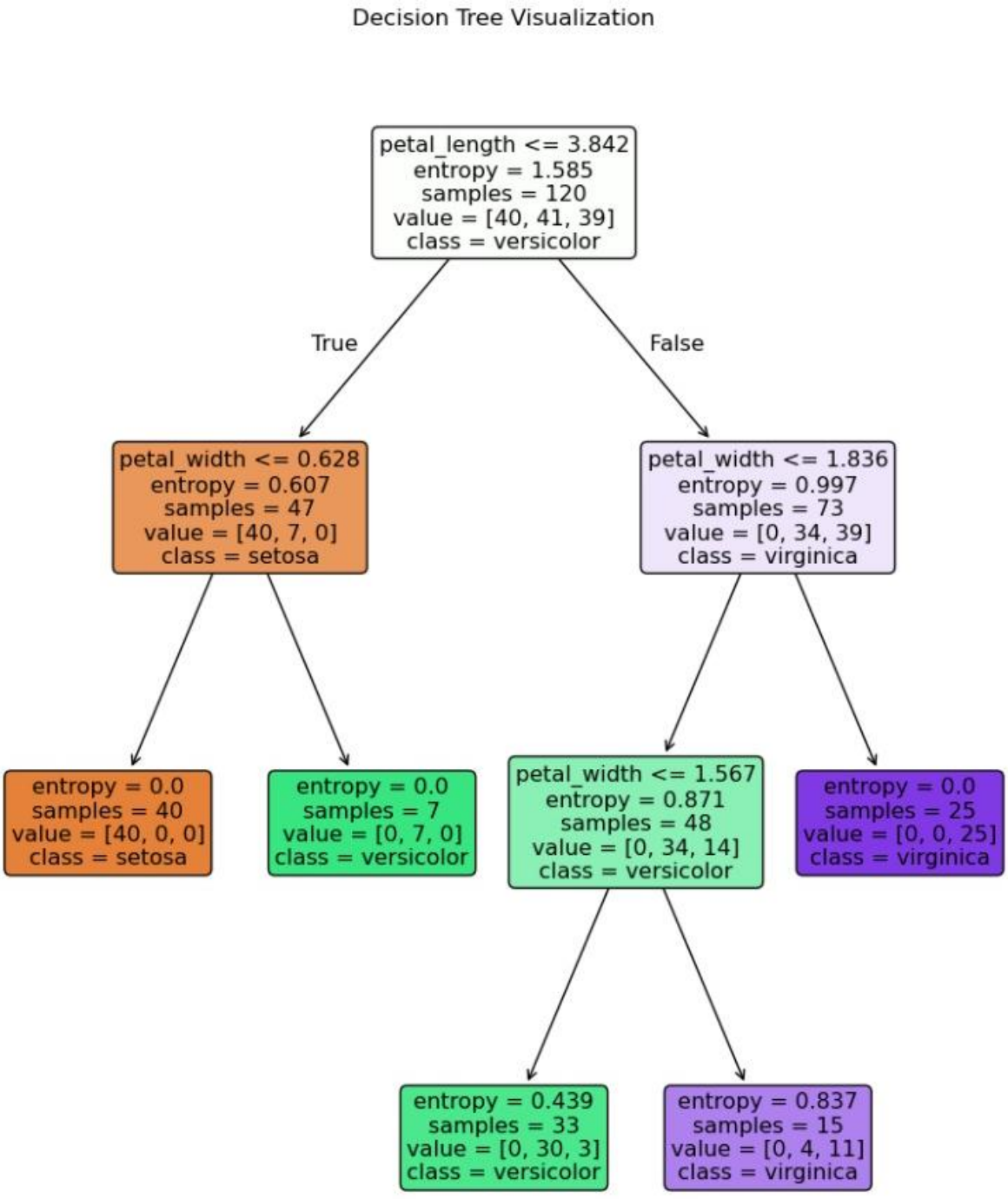
```
KNN Accuracy: 1.0
Gaussian NB Accuracy: 0.9666666666666667
```



SOURCE CODE:

```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
dataset=pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-
data/refs/heads/master/iris.csv')
x=dataset.drop('species',axis=1)
y=dataset['species']
x_train, x_test, y_train, y_test =
train_test_split(x,y,test_size=0.2,random_state=42)
dt=DecisionTreeClassifier(criterion='entropy',splitter='random',
max_depth=3)
dt.fit(x_train,y_train)
y_pred=dt.predict(x_test)
acc=accuracy_score(y_test,y_pred)
acc
class_names_list = y.unique().astype(str).tolist()
feature_names_list = x.columns.tolist()
plt.figure(figsize=(10, 12))
plot_tree(dt, filled=True,
feature_names=feature_names_list, class_names=class_names_list,
rounded=True)
plt.title("Decision Tree Visualization")
plt.show()
```

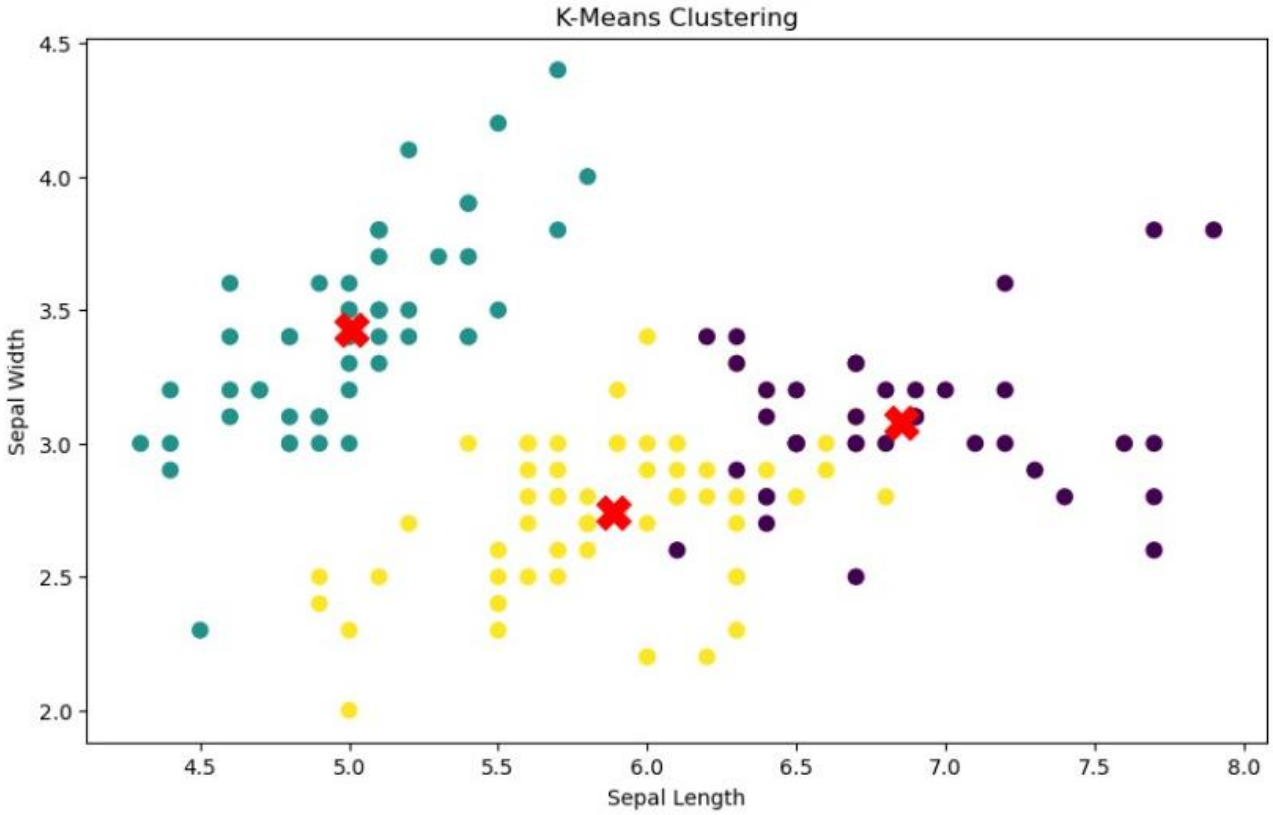
OUTPUT:


Decision Tree Visualization
petal_length <= 3.842
entropy = 1.585
samples = 120
value = [40, 41, 39]
class = versicolor
True
False
petal_width <= 0.628
entropy = 0.607
samples = 47
value = [40, 7, 0]
class = setosa
petal_width <= 1.836
entropy = 0.997
samples = 73
value = [0, 34, 39]
class = virginica
entropy = 0.0
samples = 40
value = [40, 0, 0]
class = setosa
entropy = 0.0
samples = 7
value = [0, 7, 0]
class = versicolor
petal_width <= 1.567
entropy = 0.871
samples = 48
value = [0, 34, 14]
class = versicolor
entropy = 0.0
samples = 25
value = [0, 0, 25]
class = virginica
entropy = 0.439
samples = 33
value = [0, 30, 3]
class = versicolor
entropy = 0.837
samples = 15
value = [0, 4, 11]
class = virginica

SOURCE CODE:

```
#k-means Clustering
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
dataset =
pd.read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-
data/master/iris.csv")
x = dataset.drop('species', axis=1).values
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(x)
cl = kmeans.labels_
ce = kmeans.cluster_centers_
plt.figure(figsize=(10,6))
plt.scatter(x[:,0], x[:,1], c=cl, cmap='viridis', s=50)
plt.scatter(ce[:,0], ce[:,1], c='red', marker='X', s=250)
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("K-Means Clustering")
plt.show()
```

OUTPUT:


K-Means Clustering
Sepal Width
Sepal Length
4.5
4.0
3.5
3.0
2.5
2.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0

SOURCE CODE:

```
#CNN
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.reshape((60000, 28, 28, 1)) / 255.0
x_test = x_test.reshape((10000, 28, 28, 1)) / 255.0
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28,
1)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.1)
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test accuracy:", test_acc)
```

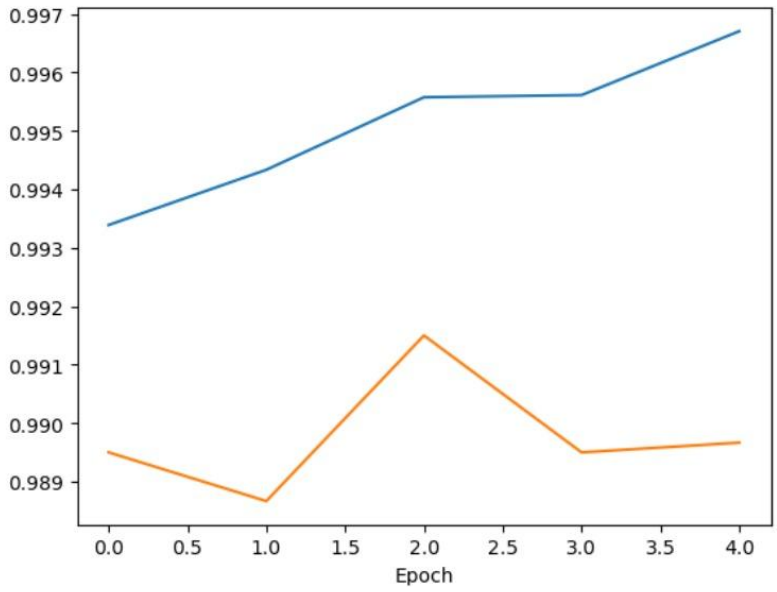
```
import matplotlib.pyplot as plt
history = model.fit(x_train, y_train, epochs=5, batch_size=64,
validation_split=0.1)
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test accuracy:", test_acc)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')

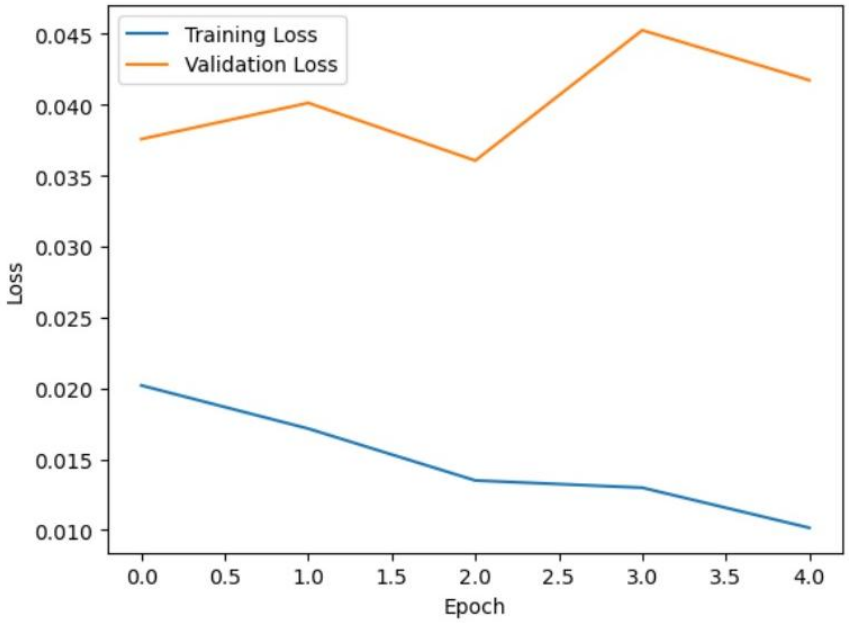
# Plot training and validation loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

OUTPUT:

```
Epoch 1/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 22s 22ms/step - accuracy: 0.9434 - loss: 0.1922 - val_accuracy: 0.9812 - val_loss: 0.0592
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 18s 21ms/step - accuracy: 0.9833 - loss: 0.0534 - val_accuracy: 0.9842 - val_loss: 0.0499
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 19s 22ms/step - accuracy: 0.9874 - loss: 0.0392 - val_accuracy: 0.9870 - val_loss: 0.0385
Epoch 4/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 18s 22ms/step - accuracy: 0.9902 - loss: 0.0307 - val_accuracy: 0.9897 - val_loss: 0.0386
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 19s 22ms/step - accuracy: 0.9924 - loss: 0.0242 - val_accuracy: 0.9912 - val_loss: 0.0312
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9915 - loss: 0.0274
Test accuracy: 0.9915000200271606
```

```
Epoch 1/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 18s 21ms/step - accuracy: 0.9934 - loss: 0.0202 - val_accuracy: 0.9895 - val_loss: 0.0376
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 18s 21ms/step - accuracy: 0.9943 - loss: 0.0172 - val_accuracy: 0.9887 - val_loss: 0.0401
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 20s 23ms/step - accuracy: 0.9956 - loss: 0.0135 - val_accuracy: 0.9915 - val_loss: 0.0361
Epoch 4/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 18s 22ms/step - accuracy: 0.9956 - loss: 0.0130 - val_accuracy: 0.9895 - val_loss: 0.0453
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 17s 20ms/step - accuracy: 0.9967 - loss: 0.0102 - val_accuracy: 0.9897 - val_loss: 0.0417
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9901 - loss: 0.0368
Test accuracy: 0.9901000261306763
[4]: Text(0.5, 0, 'Epoch')
```



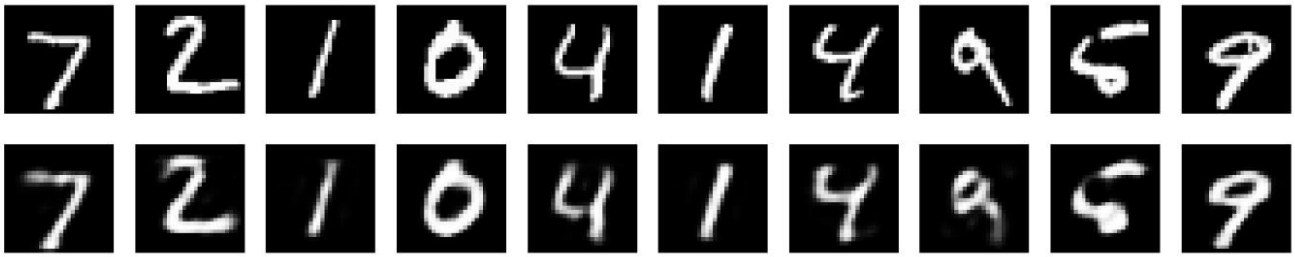

Training Loss
Validation Loss
0.045
0.040
0.035
0.030
0.025
0.020
0.015
0.010
Loss
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch

SOURCE CODE:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt (x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.reshape(60000, 784) / 255.0
x_test = x_test.reshape(10000, 784) / 255.0
input_img = tf.keras.Input(shape=(784,))
encoded = layers.Dense(32, activation='relu')(input_img)
decoded = layers.Dense(784, activation='sigmoid')(encoded)
autoencoder = models.Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
autoencoder.fit(
    x_train, x_train,epochs=10, batch_size=256, shuffle=True,
    validation_data=(x_test, x_test))
decoded_imgs = autoencoder.predict(x_test)
n = 10
plt.figure(figsize=(20, 4))
for i in range(n):
    plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
    plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.show()
```

OUTPUT:

```
Epoch 1/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - loss: 0.2744 - val_loss: 0.1883
Epoch 2/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - loss: 0.1704 - val_loss: 0.1536
Epoch 3/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - loss: 0.1447 - val_loss: 0.1346
Epoch 4/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - loss: 0.1293 - val_loss: 0.1220
Epoch 5/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - loss: 0.1187 - val_loss: 0.1133
Epoch 6/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.1113 - val_loss: 0.1072
Epoch 7/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - loss: 0.1060 - val_loss: 0.1026
Epoch 8/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - loss: 0.1021 - val_loss: 0.0992
Epoch 9/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - loss: 0.0992 - val_loss: 0.0969
Epoch 10/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - loss: 0.0972 - val_loss: 0.0952
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```



SOURCE CODE:

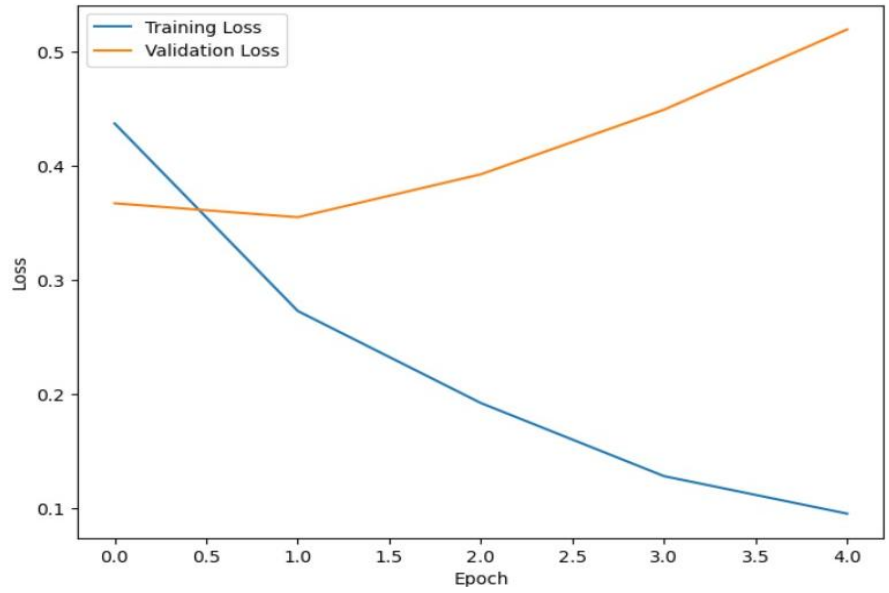
```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing import sequence
import matplotlib.pyplot as plt
max_features = 10000
maxlen = 100
(x_train, y_train), (x_test, y_test) =
imdb.load_data(num_words=max_features)
x_train = sequence.pad_sequences(x_train, maxlen=maxlen)
x_test = sequence.pad_sequences(x_test, maxlen=maxlen)
model = models.Sequential([
    layers.Embedding(max_features, 128, input_length=maxlen),
    layers.LSTM(128),
    layers.Dense(1, activation='sigmoid')])l
model.compile(optimizer='adam',
     loss='binary_crossentropy',
    metrics=['accuracy'])
history = model.fit( x_train, y_train,epochs=5,
     batch_size=32,
    validation_split=0.1)
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test accuracy:", test_acc)
```

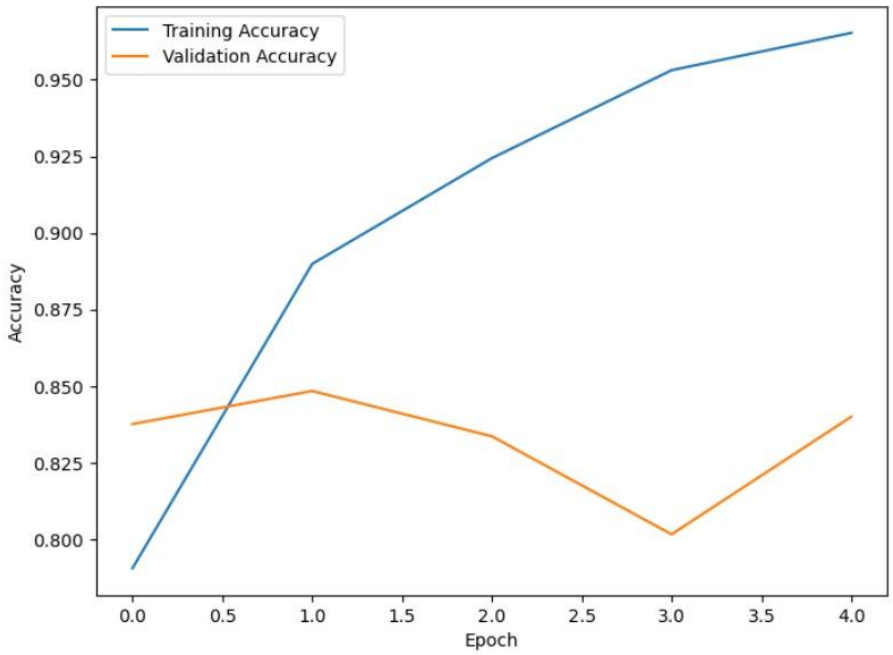
```
# Plot Loss
plt.figure(figsize=(8, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()

# Plot Accuracy
plt.figure(figsize=(8, 6))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

OUTPUT:

```
Epoch 1/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 84s 115ms/step - accuracy: 0.7905 - loss: 0.4374 - val_accuracy: 0.8376 - val_loss: 0.3675
Epoch 2/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 79s 112ms/step - accuracy: 0.8899 - loss: 0.2730 - val_accuracy: 0.8484 - val_loss: 0.3552
Epoch 3/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 77s 110ms/step - accuracy: 0.9244 - loss: 0.1924 - val_accuracy: 0.8336 - val_loss: 0.3929
Epoch 4/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 79s 112ms/step - accuracy: 0.9531 - loss: 0.1283 - val_accuracy: 0.8016 - val_loss: 0.4495
Epoch 5/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 78s 111ms/step - accuracy: 0.9653 - loss: 0.0954 - val_accuracy: 0.8400 - val_loss: 0.5198
782/782 ━━━━━━━━━━━━━━━━━━━━ 33s 42ms/step - accuracy: 0.8363 - loss: 0.5121
Test accuracy: 0.8362799882888794
```





SOURCE CODE :

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
df = pd.read_csv(r"E:\breastcancer.csv")
print(df[["id", "diagnosis", "radius_mean", "texture_mean"]].head())
df["diagnosis"] = df["diagnosis"].map({"M": 1, "B": 0})
X = df.drop(["diagnosis", "id", "Unnamed: 32"], axis=1)
y = df["diagnosis"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
img = plt.imshow(cm)
plt.title("Random Forest - Breast Cancer")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar(img)
plt.show()
```

OUTPUT:

```
        id diagnosis  radius_mean  texture_mean
0   842302         M        17.99         10.38
1   842303         B        20.57         17.77
2   842304         M        19.69         21.25
3   842305         B        11.42         20.38
4   842306         M        20.29         14.34
Accuracy: 0.9
Confusion Matrix:
[[0 1]
 [0 9]]
```

